Confluent Platform SQL Deployment via Flink SQL Client — Row-Driven PTF UDFs

> This example deploys **two row-driven PTF UDFs** (source in examples/ptf_udf_row_driven/java/) by submitting SQL statements through the **Flink SQL Client** running directly on the JobManager pod: `UserEventEnricher` (set semantics, stateful per-user session tracking) and `OrderLineExpander` (row semantics, stateless one-to-many expansion). Both UDFs ship in the same uber JAR and run as two independent long-running Flink jobs in the same session cluster.

Table of Contents

- 1.0 Overview
  - 1.1 How this differs from the other deployment paths
- 2.0 How it works
  - 2.1 JAR delivery
  - 2.2 Kafka connector
  - 2.3 Statement flow
- 3.0 Prerequisites
- 4.0 How to run
  - 4.1 Deploy
  - 4.2 Monitor
  - 4.3 Tear down
- 5.0 Resources

1.0 Overview

1.1 How this differs from the other deployment paths

Aspect	Confluent Platform SQL Client (this example)	Confluent Cloud Terraform
Where it runs	Confluent Platform on Minikube	Confluent Cloud
How SQL is submitted	`sql-client.sh -f` on the JobManager pod	`confluent_flink_statement` Terraform resources
UDF JAR delivery	`kubectl exec` to Flink pods	`confluent_flink_artifact` (uploaded to CC)
Entry point	`make deploy-cp-ptf-udf-row-driven`	`make deploy-cc-ptf-udf-row-driven`
Requires code compilation	✅ (Java + Gradle for UDF JAR)	✅ (Java + Gradle for UDF JAR)
Statement lifecycle	Managed by Flink session cluster	Managed by Terraform state

Aspect	Confluent Platform SQL Client (this example)	Confluent Cloud Terraform
Same codebase for both?	✅ (**same Java UDF code**, different Terraform vs SQL Client for deployment)	✅ (**same Java UDF code**, different Terraform vs SQL Client for deployment)

2.0 How it works

2.1 JAR delivery

On Confluent Platform there is no artifact store — the JAR must be physically present on the Flink pods.

The deploy script copies the uber JAR (built from examples/ptf_udf_row_driven/java/) to `/opt/flink/usrlib/user-event-enricher.jar` on every JobManager and TaskManager pod using `kubectl exec`. Both `CREATE FUNCTION ... USING JAR` statements (`user_event_enricher` and `order_line_expander`) reference this same pod-local path, since both PTFs are bundled in the same uber JAR.

2.2 Kafka connector

The CP Flink base image does not include the Kafka SQL connector. The FlinkDeployment uses two **init containers** to assemble `/opt/flink/lib/` at pod startup:

1. **copy-stock-lib** — copies the stock Flink lib JARs from the image into a shared `emptyDir` volume
2. **fetch-kafka-connector** — downloads `flink-sql-connector-kafka` from Maven Central into the same volume

The main Flink container then mounts this volume at `/opt/flink/lib/`, so the Kafka connector is on the classpath when the JVM starts.

2.3 Statement flow

The script pre-creates **all four Kafka topics**, then executes all SQL for **both** PTF UDFs in a single `sql-client.sh -f` session on the JobManager pod. Both UDFs are bundled in the same uber JAR, so a single pod-local JAR path feeds both `CREATE FUNCTION` statements.

```
┌──────────────────────────────────────────────────────────────────────────────┐
│  Pre-step: kafka-topics --create user_events, enriched_events,
│                                  orders, orders_expanded
│
│
│  ── UDF 1: UserEventEnricher (set semantics) ──
│
│  Step  1:  DROP TABLE IF EXISTS user_events                    → OK
│
│  Step  2:  CREATE TABLE user_events (... WITH kafka ...)       → OK
```

```
  │
  │  Step  3:  INSERT INTO user_events VALUES (sample data)        →
submitted │
  │  Step  4:  DROP TABLE IF EXISTS enriched_events                → OK
  │
  │  Step  5:  CREATE TABLE enriched_events (... WITH kafka)       → OK
  │
  │  Step  6:  CREATE FUNCTION user_event_enricher                 → OK
  │
  │            AS 'ptf.UserEventEnricher'
  │
  │            USING JAR '/opt/flink/usrlib/user-event-enricher.jar'
  │
  │  Step  7:  INSERT INTO enriched_events                         →
submitted │
  │            SELECT ... FROM TABLE(
  │
  │              user_event_enricher(
  │
  │                input => TABLE user_events PARTITION BY user_id
  │
  │              )
  │
  │            )
  │
  │
  │
  │  ── UDF 2: OrderLineExpander (row semantics) ──
  │
  │  Step  8:  DROP TABLE IF EXISTS orders                         → OK
  │
  │  Step  9:  CREATE TABLE orders (... WITH kafka ...)            → OK
  │
  │  Step 10:  INSERT INTO orders VALUES (sample data)             →
submitted │
  │  Step 11:  DROP TABLE IF EXISTS orders_expanded                → OK
  │
  │  Step 12:  CREATE TABLE orders_expanded (... WITH kafka)       → OK
  │
  │  Step 13:  CREATE FUNCTION order_line_expander                 → OK
  │
  │            AS 'ptf.OrderLineExpander'
  │
  │            USING JAR '/opt/flink/usrlib/user-event-enricher.jar'
  │
  │            -- same JAR path as Step 6, two functions, one artifact
  │
  │  Step 14:  INSERT INTO orders_expanded                         →
submitted │
  │            SELECT ... FROM TABLE(
  │
  │              order_line_expander(
  │
  │                input => TABLE orders
```

```
  │
  │               )
  │
  │             )           -- note: NO PARTITION BY (row semantics)
  │
  └──────────────────────────────────────────────────────────────────────────
  ────┘
```

Steps 7 and 14 are **two long-running streaming jobs** running side-by-side in the same Flink session cluster: Step 7 reads from `user_events` and writes per-user enriched output to `enriched_events`; Step 14 reads from `orders` and writes one expanded line-item row per order item to `orders_expanded`.

3.0 Prerequisites

- macOS with Homebrew or Linux with apt-get, and Docker Desktop running
- Java 21 and Gradle installed (for building the UDF JAR)
- Confluent Platform and Flink stack already deployed via:

```
make install-prereqs       # installs tooling (docker, kubectl, minikube,
helm, gradle)
make cp-up                 # Minikube → CFK Operator → Kafka + SR +
Connect + C3 + Kafka UI
make cp-watch              # watch pods come up (Ctrl+C when all Running)
make flink-up              # cert-manager → Flink Operator → CMF → Flink
session cluster
make flink-status          # verify Flink pods are Running
```

4.0 How to run

All commands are run from the **project root** (where the `Makefile` lives).

4.1 Deploy

A single target builds the UDF JAR, copies it to the Flink pods, and executes all SQL:

```
make deploy-cp-ptf-udf-row-driven
```

Behind the scenes this runs:

Step	What it does
1	`./gradlew clean shadowJar` — builds the UDF uber JAR from `examples/ptf_udf_row_driven/java/`
2	`kubectl exec` — copies the JAR to all JobManager and TaskManager pods

Step	What it does
3	`kafka-topics --create` — pre-creates Kafka topics (`user_events`, `enriched_events`, `orders`, `orders_expanded`)
4	`sql-client.sh -f` — executes all SQL statements for **both** UDFs in a single session on the JobManager pod

4.2 Monitor

Open the Flink Dashboard to see the **two running streaming jobs** (the user-event enrichment job and the order line expansion job):

```
make flink-ui              # opens http://localhost:8081
```

4.3 Tear down

To stop the running streaming jobs and drop all tables and functions:

```
make teardown-cp-ptf-udf-row-driven
```

This cancels any running Flink jobs via the Flink REST API, then submits `DROP FUNCTION` and `DROP TABLE` statements for **both** UDFs (`user_event_enricher`, `order_line_expander`) and all four tables (`user_events`, `enriched_events`, `orders`, `orders_expanded`).

5.0 Resources

- Flink SQL Client
- Flink Kafka Connector
- Create a User-Defined Function (Confluent Cloud)